



中山大學
SUN YAT-SEN UNIVERSITY

多智能体集群控制（智能车部分）

实验三：综合设计与赛道竞赛

姓名 党政扬

学号

学院 智能工程学院

专业 智能科学与技术

2026 年 7 月 6 日

摘要

本报告是实验三”综合设计与赛道竞赛”的完整实践报告，覆盖设计、联调与正式考试三个阶段。区别于前两次实验对单一算法的聚焦，本次任务要求车辆从固定的初始位置与朝向出发，在真实的赛道上完成”先经过起点连接段抵达大圈主赛道、再进行大圈循环”的完整路线，并具备满足比赛责任规则的避障能力。笔者首先核对了车辆真实的初始位置与朝向——起点并不在大圈主赛道上，而是位于场地内部、需要经过一段连接路径才能汇入大圈——据此确定了”先连接段、后主环道”的行驶逻辑，并设计了 Path 级别与段级别两层参数分离机制、弯道前主动降速策略。在避障环节，笔者依据比赛计分规则（里程封顶 2000 m，碰撞扣 30 ~ 50 m，异常停车超 10 s 扣 10 m，累计 4 次违规淘汰）设计了”责任范围内特调”的避障策略，将检测集中在自行车可控的两类场景——追尾前车与阻挡侧方汇入车辆；但受限于调试条件，现有 `run_log.csv` 为单车路径跟踪测试，避障与责任特调逻辑尚缺乏真实多车交互数据的量化验证。正式考试中”系统长时间运行后偶发停止、重启时错误回退至路线 1 默认初始位置而引发碰撞”这一问题——该现象与 `control_node()` 每次启动都无条件从 Path 1 开始、且不保存任何运行状态的代码结构直接相关。本报告对考试暴露的两个问题（未及时手动接管超车、系统偶发停止后错误重定位）的现象、根因与改进方向做了系统性复盘，作为后续迭代的直接依据。最终，笔者在 12 名参赛者中取得**第 5 名**的成绩：车辆里程跑满竞赛封顶上限 2000 m，但因上述两个问题累计 2 次违规、被扣除 286.77 m 里程，是名次表前列中扣分最多的一位，也印证了”跟得住”与”跑得稳、不犯规”之间的差距。

关键词：路线策略；分段参数；提前降速；责任范围避障；考试表现复盘

目录

1 实验目的	1
2 任务描述与总体设计思路	1
3 路线策略设计：从初始位置到”先连接、后主环道”行驶逻辑	2
4 分段参数设计	5
5 弯道前主动降速策略	5
6 障碍物避让实现：责任范围内的特调	7
7 综合调试与系统集成	10
8 综合调试阶段结果	10
9 正式考试表现分析与改进方向	11
10 设计反思与改进展望	14
11 实验总结	14

1 实验目的

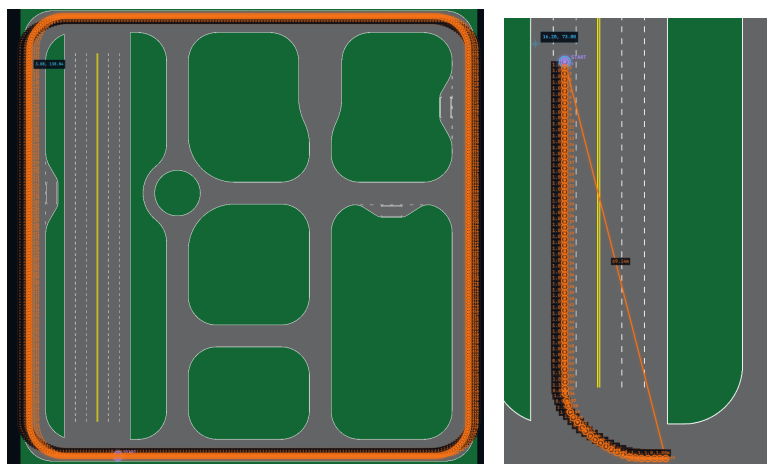
结合《(2026) 实验指导书》综合设计部分的要求，本次实验需要完成：

1. 根据车辆固定的初始位置与朝向，设计一条从起点到大圈循环的完整、安全、可自然衔接的行驶路线；
2. 针对起点连接段与大圈主赛道曲率、速度需求的显著差异，设计分层次的参数管理机制，并通过参数估算确定关键阈值；
3. 结合比赛的碰撞责任判定规则，对避障策略做针对性裁剪与特调，在保证不”犯规”的前提下尽量减少不必要的减速；
4. 在联调环境与正式考试环境下分别验证系统表现，并对二者的差异做原因分析，明确后续改进方向。

2 任务描述与总体设计思路

2.1 任务描述

本次实验的任务与前两次不同：不再是在单一路段上测试某一算法，而是要在**真实赛道**上完成一次完整的、有明确路线规划的行驶任务。赛道的实际结构（见图 1）是一个矩形的大圈主赛道，车辆的起点则位于场地内部、偏离主赛道的一侧，需要先经过一段连接路径才能汇入主赛道。据此，任务具体拆分为：Path 1（起点连接段，车辆从起始点出发，沿一段连接路径行驶，包含一段过渡直道和一个汇入弯道，到达大圈主赛道入口后完成衔接）与 Path 2（大圈主赛道循环，车辆汇入后沿矩形主环道循环行驶，包含多个直道与弯道交替的路段）。与此同时，系统还需具备满足比赛规则的基本避障能力：既不能对负有责任的碰撞不作为，也不必为不属于自己责任范围的接触过度谨慎。



(a) Path 2: 大圈主赛道整体路径点 (b) Path 1: 起点连接段路径点

图 1: 赛道整体路径点布局

将两图并置对照，可以看清整条赛道的几何关系：图 1(b) 的 Path 1 连接段起于场地内部的出生点、只有一段过渡直道加一个汇入弯道，是一次性的”接入”通道；而图 1(a) 的 Path 2 才是矩形大圈主环道，是需要反复循环计圈的主赛道。二者在坐标 (36.3, 3.0) 处首尾相接——Path 1 的终点正是 Path 2 的入口。正是这种”车辆起点不在主赛道上、需先经连接段汇入”的结构，直接决定了后文”先连接段、后大圈循环”的行驶逻辑与分段方式。

2.2 总体设计思路

拿到任务后，笔者首先面临的是一个策略级别的问题：**是否可以用同一套参数跑完 Path 1 和 Path 2**？笔者最初以为两段的弯道曲率会有明显差异，于是对全部路径点做了一次曲率估算，结果发现**赛道上所有弯道段的转弯半径其实都接近** 12.5 ~ 12.7 m（包括 Path 1 唯一的汇入弯道），并没有”急弯”与”缓弯”之分——这一开始与笔者的直觉不符。真正的差异在于**直道长度**：Path 1 的连接直道仅约 51 m，而 Path 2 各段长直道长度在 105 ~ 126 m（约为 Path 1 的 2~2.5 倍，个别收尾短段除外）。直道越长，车辆在恒定加速度下能够积累到的入弯速度就越高，Path 2 的车辆往往以接近 24 m/s 的速度逼近弯道，而 Path 1 受限于较短的直道，入弯速度天然低不少。也就是说，两段路径需要不同参数的根本原因不是”弯道有多急”，而是”车辆能积攒多快的速度冲向同样急的弯道”——车速越高，需要越早、越强地介入降速与转向修正。因此，设计的第一个决策是：**Path 1 与 Path 2 的参数集完全分离**，两段使用不同的预瞄系数、提前降速距离和 Stanley 增益，通过代码在路线切换时自动加载对应参数（第 4 节）。而在参数之前，更根本的问题其实是”路线本身该怎么走”——这是下一节要展开的内容。

路线策略、分段参数、弯道降速与责任范围避障并不是彼此独立的模块，而是沿着”初始位姿判定 → 路径配置 → 参数加载 → 弯道前降速 → 责任范围避障”这一数据流依次叠加的修正环节——这也是本报告按照”路线 → 参数 → 降速 → 避障”顺序展开的原因，每一节的方案都建立在前一节的结论之上。

3 路线策略设计：从初始位置到”先连接、后主环道”行驶逻辑

3.1 初始位置与朝向核实

设计路线之前，笔者首先核实了车辆每次上电或复位后的**真实初始状态**，而不是想当然地假设”车辆一开始就在大圈主赛道上”。参考赛道整体布局（图 1）可以看到，循环计圈的 START 标记位于矩形主赛道底边，而车辆真正的**出生点**（图 2(a)）另在起点连接段上、场地内部，需要先沿一小段 Path 1 连接路径行驶一段距离，才能汇入外围的矩形大圈主赛道。这与笔者最初”出生点就在大圈入口”的理解不同，是本节路线设计的关键前提。

图 2(a) 中车头方向与连接段初始切线的夹角接近于零，这一观察是后续所有路线决策的出发点：**车辆的物理初始状态”长”在”起点连接段上，离大圈主赛道本身还有一段距离**。这一前提在真实日志里也留下了直接痕迹：分析 run_log.csv 时，笔者发现 Path 1 汇入弯道人弯瞬间（约 $t = 8.3\text{ s}$ ）记录到一段幅值高达 17 m 的横向误差瞬态（详见第 5.4、8.1 节）——这并非车辆真的偏出 17 m，而是车辆当时的实际位置与 Path 1 弯道段所假设的起点存在明显偏差、局部最近点尚未

收敛所致，恰恰是”物理初始状态与路段假设不完全对齐”的量化体现。如果忽视这一前提、在代码里直接假设车辆一启动就应该追踪 Path 2（大圈主赛道）的路径点，就会出现车辆朝向与目标路径夹角过大、纯追踪/St Stanley 控制器给出的转向指令剧烈摆动的问题。

3.2 路线分段设计

在明确初始状态与赛道结构后，笔者将整条行驶任务按照曲率与功能定位拆分为两级、共 11 个逻辑段：Path 1（起点连接段）拆分为 1 段过渡直道 + 1 段汇入弯道，直道用于让车辆从静止加速到接近汇入大圈所需的速度，弯道用于完成从连接路径到主赛道切线方向的姿态转换，终点即为大圈主赛道入口；Path 2（大圈主赛道循环）拆分为 9 段（编号 1-9），对应矩形主赛道的四段长直道、四个转角弯道，外加一段收尾短直道，直道与弯道交替排布，以 `cycle_order = ['1','2',...,'9']` 的顺序首尾循环。

Path 1 与 Path 2 的整体路径点布局已在图 1 中并置展示，二者合在一起完整覆盖了共 11 个逻辑段（Path 1 的 2 段 + Path 2 的 9 段）的真实几何布局。经对各段 JSON 路径点的实测统计，Path 2 四段长直道长度分别约为 105、122、126、122 m，收尾短段约 15 m；四个弯道段弧长均约 23 ~ 27 m、拟合半径 12.5 ~ 12.7 m，与 Path 1 汇入弯道（半径约 12.6 m）基本一致——这正是第 2 节”弯道半径相近、直道长度悬殊”判断的数据依据。

这样拆分的依据是：直道和弯道对速度、预瞄距离、控制增益的需求本质不同，若不分段就无法分别调参（第 4 节）；而 Path 1 / Path 2 的划分则直接对应了”起点连接段”与”大圈主赛道”两种不同曲率、不同比赛职责的路段类型——前者是一次性的过渡任务，后者才是需要反复循环的主赛道。

3.3 ”先连接段，后主环道循环”行驶逻辑

有了初始状态与分段方案后，路线的核心逻辑就自然确定了：车辆先沿 Path 1 连接段从出生点行驶到大圈主赛道入口，再进入 Path 2 大圈循环反复绕行。笔者用 `analyse.py` 对真实 `run_log.csv` 还原出的实际行驶轨迹也印证了这一逻辑：车辆从出生点先沿一段连接路径向下、汇入矩形大圈，随后沿大圈连续绕行了约 3 圈；逐帧统计其横向误差可见，弯道处误差普遍偏大、直道段几乎可忽略，Path 1 入弯瞬间还出现过一次约 17 m 的初始定位瞬态（见第 3.1、5.4 节及表 2）。

采用”先完整跑完连接段、再自然过渡到主赛道”，主要是因为它同时匹配三件事：车辆初始朝向本来就与 Path 1 切线一致，避免了一启动就横向切入 Path 2 带来的大角度修正；Path 1 与 Path 2 的参数集可按路线阶段自然切换，短直道低入弯速度与长直道高入弯速度不再混用同一套预瞄和 Stanley 增益；同时，汇入阶段的侧向冲突与循环阶段的同车道追尾正好对应后续责任范围避障的两类重点场景。综合来看，这不是孤立的路线选择，而是由初始状态、参数体系与比赛责任规则共同决定的方案。

3.4 路线拼接实现

检查两段路径的原始坐标点后，笔者确认 Path 1 弯道终点 (36.3, 3.0) 与 Path 2 第 1 段起点 (36.3, 3.0) 完全重合——两段路径本就是同一条连续曲线的延伸，物理上不存在需要”桥接”的空间

间隙。既然如此，为什么代码里仍然把 Path 2 的第 1、2 段单独作为一次性的”前导段”（Preamble）运行，而不是直接从第 1 段起就按 `cycle_order` 循环？原因是圈速循环的”起点/终点”被定义在第 3 段（`cycle.index('3')`），第 1、2 段在几何上属于大圈的一部分，但在计圈逻辑上被视为”从连接段驶入大圈后、正式开始计圈之前”的一次性过渡：程序先完整跑完 Path 1，紧接着顺路跑一遍第 1、2 段（此时车辆仍在物理上连续行驶，没有任何跳变），真正的循环计数从第 3 段开始，此后按 $3 \rightarrow 4 \rightarrow \dots \rightarrow 9 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$ 重复。

```

1 def _run_path2_loop(self):
2     seg_dict = {s['name']: s for s in self.path2_segments}
3     cycle = self.path2_cycle
4     loop_start_idx = cycle.index('3')
5     for i in range(loop_start_idx):          # preamble: 1 -> 2 -> 3
6         self._run_segment(seg_dict[cycle[i]], seg_dict[cycle[i + 1]])
7     idx = loop_start_idx
8     while True:
9         next_idx = (idx + 1) % len(cycle)
10        self._run_segment(seg_dict[cycle[idx]], seg_dict[cycle[next_idx]])
11        idx = next_idx

```

Listing 1: 路线切换与 Path 2 前导段设计（main.py 节选）

把车辆从起步到稳定循环的三个关键瞬间并置来看（图 2），可以串起完整的实测行驶过程：(a) 在出生点，车头朝向与黄色车道中线（连接段初始切线）基本一致，为沿 Path 1 平顺起步创造了条件；(b) 完成 Path 1 后经前导段汇入 Path 2 大圈主赛道的瞬间，车身姿态已与车道线基本平行，没有出现汇入时常见的横向急修正；(c) 在大圈主赛道弯道段过弯时，车身贴合车道内侧、未明显偏出车道。三张实拍从朝向、汇入到过弯逐段印证了”先连接段、后大圈”的路线逻辑与提前降速策略在真实仿真环境中的有效性。



(a) 出生点 (b) 进入外圈：平滑汇入大圈 (784.41 m) (c) 外圈过弯：贴合车道外侧 (192.43 m)

图 2: 实测行驶过程的三个关键瞬间（Unity 实测截图）

4 分段参数设计

4.1 为什么需要分段参数

前两次实验中，Path 1 和 Path 2 共用一套参数。但如第 2 节所述，两段路径的弯道半径其实相近（均约 12.5 ~ 12.7 m），真正需要区别对待的是**直道长度差异带来的人弯速度差异**：Path 2 的直道（105 ~ 126 m）远长于 Path 1（51 m），车辆在 Path 2 上会积累到更高的人弯速度。如果用同一套（偏保守的）参数应对两段路径，Path 2 直道的时间表现会明显拉胯；如果用同一套（偏激进的）参数，Path 1 上车辆虽然入弯速度本就较低，但过短的提前降速距离仍可能让残余误差偏大。这说明“用一套全局参数驾驭入弯速度差异悬殊的路段”是不合理的，需要分段独立配置。

4.2 两层分段设计

笔者引入了两个层次的参数分离机制。第一层是 **Path 级别参数集**：维护 `path1_params` 和 `path2_params` 两个独立字典，路线切换时调用 `_apply_path_params()` 加载。实际取值压缩列于表 1。

表 1: Path 级别参数集实际取值

路径	m_v	k_{pp}	$\min L_d$	提前降速距离	v_{entry}	k_e/k_s
Path 1	20.0	0.85	3.0	5.0	15.0	2.0 / 1.0
Path 2	25.0	0.90	4.0	20.0	15.0	3.0 / 1.0

第二层是**段级别速度覆盖**：`master_config.json` 中每一段可单独配置 "speed" 字段，运行时覆盖 Path 级别的 m_v 。实际部署中，Path 2 的 9 段以直道（25.0 m/s）与弯道（14.0 m/s）交替排布，四段长直道长度在 105 ~ 126 m 之间，四个弯道半径均约 12.5 ~ 12.7 m。

5 弯道前主动降速策略

5.1 问题的发现

在分段速度配置完成后，仍然出现了一个顽固问题：弯道段 JSON 虽然设置了 14.0 m/s，但车辆在直道末尾以 20 ~ 25 m/s 全速冲到弯道起点，刹停到 14.0 m/s 需要若干帧，这几帧中车辆已经高速进入弯道，导致进弯初始误差很大，甚至偶发“冲出弯道”的危险情况。关键诊断是：**段速度切换是阶跃式的，而物理减速需要时间**——只在段边界降速是不够的，必须在进入弯道 **之前**就开始降速。

5.2 `curve_approach_dist` 取值依据

提前降速距离的取值不是拍脑袋决定的，而是与第 2 节的观察直接挂钩：Path 1 的连接直道仅约 51 m，车辆在其上按 20 m/s² 的加速度爬升，实际能达到的入弯速度有限，5 m 的提前降速距离

已经足够把速度从当前值收敛到 $v_{entry} = 15 \text{ m/s}$ ；而 Path 2 的直道长达 $105 \sim 126 \text{ m}$ ，车辆有充分空间加速到接近 25 m/s 的目标速度，若仍沿用 5 m 的提前量，同样的加速度下根本来不及减到 15 m/s ，因此需要把 `curve_approach_dist` 放大到 20 m ，让速度有足够的距离线性回落。这一推算笔者未做逐值扫描实验（受限于单车调试时间），而是先用运动学关系估算量级，再通过实际部署后的运行日志（第 5.4 节）反过来检验取值是否合适。

5.3 解决方案：弯道前线性降速

在直道段控制循环中，当检测到下一段为弯道且距当前段终点的距离小于 `curve_approach_dist` 时，启用线性降速逻辑，将目标速度从当前 v 线性插值到弯道入口速度 v_{entry} ：

```
1 def _apply_curve_approach_brake(self, v, w):
2     dist_to_end = distance_to_segment_end()
3     if dist_to_end < self.curve_approach_dist:
4         t = dist_to_end / self.curve_approach_dist
5         v_brake = self.curve_entry_v + t * (self.seg_v - self.curve_entry_v)
6         v = min(v, v_brake)
7     return v, w
```

Listing 2: 弯道前提前降速实现（main.py 节选）

该逻辑对应的速度剖面很直观（由 `path1_params` / `path2_params` 的实际取值即可推算）：直道段先维持段目标速度，进入 `curve_approach_dist` 范围后目标速度沿距离线性回落到 $v_{entry} = 15 \text{ m/s}$ ——Path 1 在最后 5 m 、Path 2 在最后 20 m 内完成回落，从而把”段边界阶跃降速”改造为”提前、平滑降速”。

5.4 部署后实测：各弯道段的真实横向误差

由于单人调试很难对 `curve_approach_dist` 做完整的多值对照实验，笔者转而直接分析部署后一次完整跑动（Path 1 + Path 2 连续约 3 圈）的 `run_log.csv`，逐段统计 Stanley 控制律中实时记录的有符号横向误差 e （该字段在直道纯追踪阶段恒为 0，只在弯道段有意义），结果如表 2 所示。需要特别说明的是：Path 1 汇入弯道在入弯瞬间（约 $t = 8.3 \text{ s}$ ，共 20 帧）出现了一段幅值约 17 m 的初始定位瞬态（成因见第 3.1、8.1 节），显然不代表车辆真实偏移，因此表中 Path 1 弯道一栏统计的是剔除该入弯瞬态后的稳态误差，瞬态本身另行单列讨论。

表 2: 部署后实测：各弯道段稳态横向误差统计（来自真实 run_log.csv, logged Stanley e）

路段	弯道段	平均速度 (m/s)	Mean $ e $ (m)	Max $ e $ (m)	RMS (m)
Path 1	seg 2	12.8	0.397	1.602 [†]	0.551
Path 2	seg 2	12.1	0.553	1.263	0.676
Path 2	seg 4	12.1	0.565	1.517	0.733
Path 2	seg 6	11.4	1.017	2.314	1.220
Path 2	seg 8	12.6	0.328	1.073	0.418

[†] 已剔除 Path 1 入弯瞬间约 20 帧的初始定位瞬态（该瞬态原始 max $|e| \approx 17.4$ m, 非真实偏移）。

表 2 证实了提前降速确实起了作用：所有弯道段的平均速度都被稳定收敛到 11.4 ~ 12.8 m/s 附近（低于 $v_{entry} = 15$ m/s 的设定上限），没有出现“全速冲入弯道”的情况。但数据同时暴露了一个笔者最初没有预料到的问题：**Path 2 的 seg 6 弯道横向误差明显大于其余弯道**——其稳态最大误差达到 2.31 m，是 Path 1 汇入弯道稳态误差（1.60 m）与 seg 8（1.07 m）的约 1.4–2.2 倍。由于全赛道弯道半径基本一致（第 2 节），这一差异不能用曲率来解释，更可能与车辆在连续多个弯道之后的姿态累积误差、或个别弯道前直道（如 126 m 的第 5 段直道）末端速度控制的细微差别有关，是笔者在报告完成后仍需要继续排查的问题，而不是一个已经彻底解决、可以直接写“完美消除”的结论。将表 2 中各弯道的 Mean/Max/RMS 误差横向对比也能看出，seg 6 的三项指标都明显高出一截。

提前降速部署后车辆在大圈主赛道弯道段的实测过弯画面已在图 2(c) 给出，可与表 2 中误差较小的弯道段对照：车身贴合车道内侧行进，直观印证了降速策略在多数弯道上的有效性。

6 障碍物避让实现：责任范围内的特调

6.1 基础避障算法

本次实验的基础避障沿用笔者在**实验二**中已详细设计并验证的方案——以**路径切线方向**（而非车辆瞬时朝向）为参考系投影邻车、再按纵向距离做分级线性减速；其几何关系与完整推导详见实验二（“分级线性减速策略示意”一图及对应公式），此处仅作必要回顾、**不再重复其几何示意图**。检测流程为：将邻车坐标投影到自车当前的路径切线方向，分离出纵向距离 d_{fwd} （沿路径前方）和横向距离 d_{lat} （垂直于路径）；若 $|d_{lat}| > d_{lat,th} = 3.0$ m 视为不在同一车道，忽略；若 $d_{stop} < d_{fwd} \leq d_{safe}$ ，按线性比例将目标速度从 v 衰减至 0：

$$v_{obs} = v \cdot \frac{d_{fwd} - d_{stop}}{d_{safe} - d_{stop}} \quad (1)$$

若 $d_{fwd} \leq d_{stop}$ ，目标速度置零。也就是说，只有落入自车正前方、且横向偏移小于 $d_{lat,th}$ 的邻车才会触发减速，检测范围随距离分为 d_{safe} （开始线性减速）与 d_{stop} （完全停车）两层。

```

1 def _check_obstacle(self, v, w):
2     path_yaw = self.path_yaw[self.nearest_idx]
3     for nb in self.udp_client.get_neighbor_vehicle_state():

```

```

4      d_fwd, d_lat = project_to_path_frame(nb, path_yaw)
5      if d_fwd <= 0 or abs(d_lat) > self.obs_lateral_thresh:
6          continue
7      if d_fwd <= self.obs_stop_dist:
8          v = 0.0
9      elif d_fwd <= self.obs_safe_dist:
10         ratio = (d_fwd - self.obs_stop_dist) / (self.obs_safe_dist - self.
obs_stop_dist)
11         v = min(v, v * ratio)
12     return v, w

```

Listing 3: 障碍物检测与减速实现（main.py 节选，基础版本）

笔者刻意将避障限制为”仅纵向减速/停车”，不改变转向角，理由是：转向绕障需要对障碍物形状、路边界有更精确的感知，仅靠前方距离估计贸然转向反而可能撞上路侧；而纵向制动是最保守、最可靠的应对方式，也最容易在比赛规则下被判定为”已尽到避让义务”。

6.2 比赛计分规则驱动的避障范围特调

在完成基础避障算法后，笔者进一步思考了一个问题：**是否有必要把避障做成”全向检测”，覆盖所有可能发生接触的方向？**结合《实验指导书》给出的竞赛计分规则重新审视后，笔者的结论是否定的。计分规则的核心内容是：单车全程里程最高计入 2000 m；驶入非道路区域扣 20 m；发生碰撞扣 30 ~ 50 m；异常停车超过 10 s 扣 10 m；累计违规达到 4 次直接淘汰。这些规则里唯一与”避障”直接相关的两条是”碰撞扣分”与”异常停车扣分 / 淘汰”，而自车真正能够通过感知与控制主动避免的碰撞场景，仔细梳理后其实只有两类：一是本车在同车道内追尾正前方车辆（本车速度、跟车距离完全由自己控制）；二是本车的行驶行为迫使起点连接段与大圈主赛道汇合区域内正在汇入的侧方车辆被迫减速或停车（本车是否让行、何时让行同样由自己决定）。除此之外的接触——例如被后车追尾、双方都在正常行驶中的擦碰、被侧方车辆主动超越时的轻微接触——本车既无法预先感知对方的全部意图，事发时也往往缺乏反应时间，属于自车控制能力之外的风险，投入额外的检测资源也很难真正规避。图 3 把上述判断可视化：只有落在”前方追尾”与”侧方来车停止”两块检测范围内的接触才属于自车责任，范围之外的接触则被有意留作不额外处理的区域。

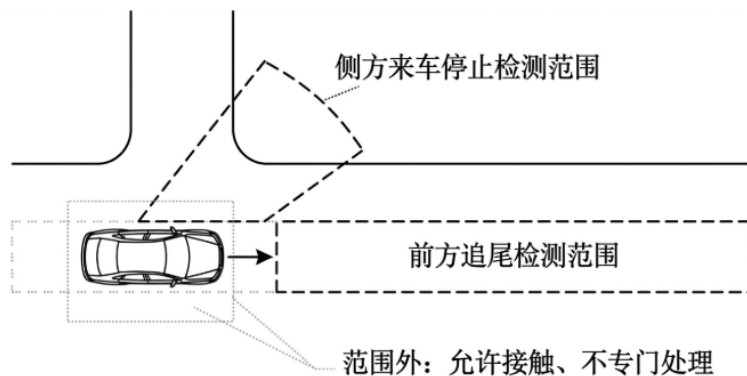


图 3: 责任范围避障特调示意：俯视图展示起点连接段与大圈主赛道的汇合区域，标注”前方追尾”与”侧方来车停止”两类责任场景的检测范围，以及范围外允许接触的区域

这一梳理实际上给出了一个明确的性价比边界：为了规避那些自车本就难以主动避免的接触类型而额外投入检测与控制资源，性价比很低，反而会增加不必要的减速、拖慢圈速、甚至因为误判而在无威胁时错误刹车。基于此，笔者对连接段汇入运行阶段的避障策略做了责任范围内的特调：前方追尾风险沿用基础算法中“ $d_{fwd} > 0$ 且 $|d_{lat}| < d_{lat,th}$ ”的判定条件，天然只响应处于本车正前方车道内的车辆；侧方来车停止风险仅出现在起点连接段与大圈主赛道的汇合区域（第 3 节路线拼接区间），笔者为该区间单独设计了一段窗口检测逻辑，只有在车辆位于汇合区域附近、且检测到侧方有邻车处于低速或静止状态时，才触发保护性减速；离开汇合区域后立即恢复正常速度。其余方向（如被后方来车接近、与侧方正常行驶车辆的轻微交会）不做专门处理。

该设计方案在代码层面可概括为：仅在 `_in_merge_zone()` 为真时启用侧向邻车检查；若侧向车辆横向距离小于阈值且速度低于停止阈值，则按距离比例短暂降低本车速度，离开汇合区后立即关闭该检测。

6.3 阈值取值依据与验证现状

$d_{stop} = 10\text{ m}$ 、 $d_{safe} = 30\text{ m}$ 、 $d_{lat,th} = 3.0\text{ m}$ 是最终写入 `main.py` 的参数（第 6 节代码节选）。取值的基本考虑是： d_{safe} 需要大于车辆在 Path 2 高速直道（约 25 m/s ）上按 $accel = 20\text{ m/s}^2$ 制动到安全跟车速度所需的距离，留出充分的线性减速区间； d_{stop} 则保证即便车辆未能及时响应，也能在障碍物前完全停住。需要如实说明的是：本报告用于量化分析的 `run_log.csv`（第 5.4 节）是**单车路径跟踪测试**，日志中没有出现任何邻车数据，也就没有真正触发过 `_check_obstacle` 的减速/停车分支（第 8 节表 3 的“避障触发次数”为 0 即由此而来）。也就是说， d_{stop} 、 d_{safe} 两个阈值目前只经过了运动学层面的估算和代码走查，尚未在真实多车交互场景下取得量化验证数据（例如误刹车次数、责任场景漏检次数等）；同时，上一小节的 `_check_merge_zone_side_risk` 汇合区特调目前仍停留在设计方案阶段，尚未合并进 `main.py` 主控制循环。这是本报告在诚实性上需要明确标注的局限，也是后续多车联调阶段需要优先补齐的工作。

6.4 合理性与局限性

这一特调的合理性在于：它把有限的调参与调试精力集中在真正会影响比赛成绩（判责、罚分）的两类场景上，与第 3.3 节“先连接段、后主环道”的路线逻辑也是自洽的——责任场景本身就集中在汇合区域和同车道追尾这两处，与路线结构一一对应。其局限性同样值得记录：一是该策略高度依赖“计分规则不会变化”这一假设；二是“侧方来车停止”目前只在固定的汇合区域窗口内检测，如果大圈循环中出现新的临时汇合点或多车交叉场景，需要额外扩展检测区域；三是如上一小节所述， d_{stop} 、 d_{safe} 两个阈值尚未经过真实多车数据验证，存在“实际部署后发现偏保守或偏激进都需要重新调参”的风险；四是“其余方向不做专门处理”这一设计选择虽然在当前规则下是合理的性价比取舍，但也意味着车身其他部位一旦发生接触，仍可能对车辆本身造成物理损耗或影响后续控制稳定性，这是第 9 节考试表现问题的部分诱因之一。

7 综合调试与系统集成

7.1 段切换失败问题

在系统集成测试中，出现了一个偶发性问题：车辆高速通过段终点时，由于控制频率有限（30 Hz），相邻两帧之间车辆可能已经“越过”了终点，导致基于“到终点距离 $< 1.5\text{ m}$ ”的单一判断失效，车辆错过切换直线冲出弯道。笔者为 `_segment_end()` 增加了多重检测条件，只要满足其一即切换：到段末尾路径点的欧氏距离 $d < 1.5\text{ m}$ ；最近点索引 $i_{\min} \geq N - 5$ ；车辆在末尾点附近越过了路径切线（点积变号）。

7.2 人工移动后重新定位问题

测试中还遇到一种情况：测试暂停时由键盘接管控制，将车辆移至安全点位，之后重启自动驾驶，车辆不知道该追踪哪个路径点。这是因为车辆位置发生了突变（ $> 15\text{ m}$ ），基于“最近索引附近局部搜索”的 `_update_nearest_idx()` 会严重误判。解决方案是监控位置跳变：当车辆偏离当前 `nearest_idx` 锚点过远时，触发全局搜索 `_global_nearest_search()`，在整个当前段路径点集中重新找最近点。这一机制在第 9 节的考试问题复盘中被再次提及——“它能处理”同一段内的位置跳变”，却无法处理“进程重启后连加载的段都不对”这种更彻底的状态丢失。

7.3 UDP 冷启动定位问题

实验初期还发现：程序刚启动时 UDP 接收线程尚未就绪，`get_vehicle_state()` 可能返回初始值 $(0, 0, 0)$ 。若此时立即全局搜索最近点，起始索引会被错误初始化。修复方式是在控制循环前设置最多 5 s 的等待窗口，直到收到 (x, y) 非零的首帧有效位姿后再调用 `_global_nearest_search()`；若超时，则输出警告并保留当前位姿继续运行。

8 综合调试阶段结果

本次单车联调所用的 `run_log.csv` 完整记录了车辆从起点连接段驶入大圈、连续跑完约 3 圈大圈循环的过程。以该日志共 3665 帧、控制频率稳定在 30.0 Hz 的有效行驶数据为准，核心跟踪功能的真实性能指标如表 3 所示。

表 3: 单车联调测试的真实性能指标

指标	Path 1	Path 2	备注
直道平均速度 (m/s)	23.1	23.8–23.9	目标 25.0, 未完全达到
弯道平均速度 (m/s)	12.8	11.4–12.6	目标 14.0, 提前降速生效
弯道 $\text{Mean} e $ (m)	0.397	0.33–1.02	见表 2
弯道 $\text{Max} e $ (m)	1.602	1.07–2.31	seg 6 明显偏大
段切换失败次数	0	0	多重检测条件均未触发异常
避障触发次数	0	0	单车测试无邻车数据, 未触发

联调阶段的核心跟踪功能（路线衔接、分段参数、弯道降速）总体工作正常，直道段速度虽未完全达到 25.0 m/s 的目标值（主要受加速度爬升与提前降速窗口挤占影响），但已经比较接近；弯道段除 seg 6 外误差也基本可控。但责任范围避障逻辑因为缺乏真实多车数据，实际上**完全没有被验证过**——这本身就是联调阶段的一处盲区。

8.1 分析工具本身的一个问题：全局最近点匹配导致的伪误差

在用分析脚本处理这份 `run_log.csv` 时，笔者还意外发现分析环节自身存在一处缺陷：早期版本的轨迹分析脚本 (`run_analyzer.py`) 对整条赛道做**全局**最近路径点匹配来计算横向误差，而不是像 `main.py` 的 `_update_nearest_idx()` 那样限制在“当前索引附近的局部窗口”内搜索。当车辆在段与段之间切换、或者完成一整圈回到起始段附近时，全局最近点搜索偶尔会匹配到赛道上几何距离很近、但路径顺序上相隔很远的另一个路段的点，导致个别帧被算出上百米的“横向误差”（笔者用该脚本处理本次日志时，曾观察到误报的单帧最大“误差”高达 158.65 m，显然远超物理可能性，是分析脚本本身的匹配缺陷，而非车辆真实的跑偏）。这一发现促使笔者改为直接使用 `main.py` 自身在控制循环中实时计算并记录的 Stanley 横向误差字段 `e`（该字段的最近点搜索本就限制在当前段内，不会跨段误判），才得到表 2 中真实可信的弯道误差数据。值得一提的是，第 3.1 节提到的 Path 1 入弯 17 m 瞬态与此处的 158.65 m 伪误差属于**不同性质**：前者是车辆真实位置与路段假设起点不一致导致的初始定位偏差（发生在控制器内部、会真实影响转向），后者纯粹是离线分析脚本的匹配错误（不影响车辆行为）。这一插曲也提醒笔者：**分析工具本身的正确性同样需要交叉验证**，不能因为某个脚本“跑出了数字”就默认这些数字反映真实物理情况。

9 正式考试表现分析与改进方向

联调阶段表现良好，不代表正式考试就能取得同样的效果。实际考试过程中暴露出两个在联调测试里没有充分暴露、或者说没有被足够重视的问题，直接影响了最终成绩。本节先给出比赛的最终成绩，再逐一复盘这两个问题。

9.1 比赛成绩

本次赛道竞赛共 12 名参赛者同场竞技，笔者（参赛编号 0212）最终排名第 5 名，关键计分如表 4 所示。

表 4: 正式比赛计分结果（笔者与名次表前列选手对比）

排名	编号	里程（m）	违规次数	扣除里程（m）
1	0211	2000.00	0	-0.17
2	0209	2000.00	0	-131.87
3	0210	2000.00	0	-61.67
4	0204	2000.00	2	-100.15
5	0212（笔者）	2000.00	2	-286.77
6	0216	2000.00	1	-75.30

从计分结果看，车辆里程达到了竞赛封顶上限 2000.00 m，说明路线跟踪与循环能力达标；差距主要来自**违规与扣分**。笔者累计 2 次违规、扣除 286.77 m，是名次表前列中扣分最多的一位。追溯这两次违规，直接原因是比赛过程中系统被多次暂停、重启后，车辆错误地从 Path 1 起点默认位置重新开始追踪，而此时车辆物理上已在大圈主赛道中段，错误目标点最终引发碰撞。前车阻挡是外因，重启后回退 Path 1 才是扣分的直接根源。

9.2 问题一：未及时手动接管超车，被前方慢车长时间卡住

现象：考试过程中，本车在大圈循环阶段遇到前方存在一辆车速明显低于自身目标速度的车辆。根据第 6 节的责任范围避障策略，本车检测到前车后按式 (1) 线性减速、与前车保持安全距离跟随，全程没有发生追尾，从”避障是否正确”的角度看策略本身没有失效。但问题在于：该避障策略**只有减速能力，没有变道/超车能力**，而当时比赛规则允许操作员在必要时手动接管方向盘完成超车，笔者却没有及时做出手动接管的判断，导致车辆长期跟在慢车后方以远低于目标速度的速度巡航，圈速被严重拖慢。

原因分析：笔者对”自动跟车”效果信任过高，看到车辆没有追尾、跟车过程较平顺，便误判为系统仍在正常工作；同时，当前避障策略本身只负责减速，不具备变道/超车能力，control_node() 中也没有阻塞计时或横向偏移绕行逻辑。换言之，考试当天并非”自动超车失效”，而是自动超车从未真正集成，只能依赖人工判断介入。

改进方向：一是在控制程序中加入”持续跟车减速计时”提示，若车辆因前车低速持续跟随超过阈值（如 5s），提醒操作员手动接管超车；二是将”阻塞计时 + 横向偏移”状态机合入主控制循环，超过阈值后沿路径法线施加临时偏移，绕行结束再线性回零，从而降低对人工反应速度的依赖。

9.3 问题二：多次暂停/停止后重启，错误回退至路线 1 初始位置引发碰撞

现象：考试过程中，系统因被前车阻挡等原因被**多次暂停、重启**，也出现过系统自身的偶发停止。问题在于：**每一次重启**后，车辆的控制逻辑都没有从停止前所在的赛道位置（大圈主赛道循环中的某一段）继续追踪，而是无一例外地按照**路线 1（起点连接段）**默认的**初始位置和分段**重新开始运行，与车辆当时实际所处的物理位置严重不符，于是在重新起步的过程中**反复**与赛道上的其他车辆发生碰撞——这正是本次比赛两次违规的直接来源。

原因分析：对照 main.py 的 control_node() 可知，这不是单次误判，而是**结构性缺陷**：程序每次启动或重启都会无条件先执行 Path 1，再进入 Path 2 循环，类中没有字段记录或持久化当前 Path、段号、圈数和最近索引。第 7 节的重新定位机制只能处理同一进程、同一路段内的位置跳变，无法处理“进程重启后加载的段本身就错了”。因此一旦重启发生，代码会在 Path 1 的路径点里搜索最近点，即使车辆物理上位于 Path 2 中段，也会被引向错误目标，最终造成碰撞。这类问题需要长时间运行甚至重启才能暴露，短时联调很难充分覆盖。

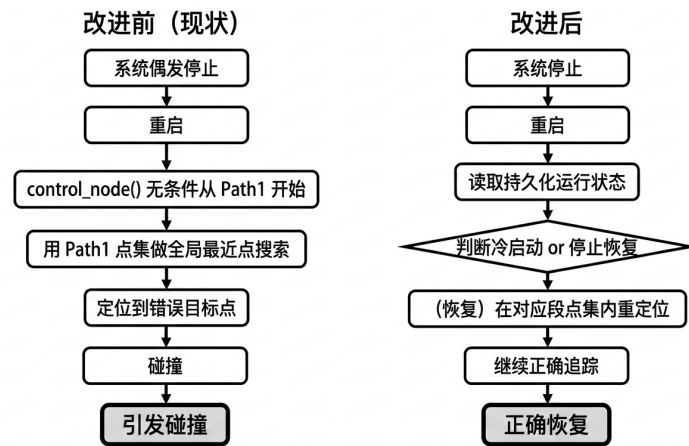


图 4: 重启定位逻辑改进前后对比示意：左侧为现有逻辑中“停止后重启”被误判为“冷启动”、状态被重置为路线 1 默认位置的错误路径；右侧为改进后基于运行状态持久化、正确区分冷启动与停止恢复的流程

图 4对比了改进前后的定位流程：左侧把“停止后重启”误当成“冷启动”，右侧则基于运行状态持久化恢复到正确路段。后续实现应集中在三点：其一，以固定周期记录当前 Path、段名、cycle 索引和 nearest_idx，形成“最近已知良好状态”；其二，区分冷启动与停止恢复，只有从未收到有效 UDP 状态时才回退到 Path 1 默认位置，否则应在记录的对应段内重新定位；其三，增加日志与看门狗机制，并在赛前做覆盖正式考试时长的压力测试，验证偶发停止后的恢复逻辑是否真正生效。

9.4 两个问题之间的关联

值得指出的是，这两个问题并非完全独立：问题一说明避障策略本身“只减速不超车”的设计局限（第 6 节末已指出），在长时间运行、多车交互的正式考试环境下被放大成了实际的成绩损失；问题二则说明系统在应对“运行时异常”这一类边界情形时，健壮性仍然不足，与第 7 节已经修复的边界问题相比，这是一类新的、发生在**系统级重启而非单车控制细节**层面的问题。两者共同提示：

仅仅让核心控制算法（跟踪、避障）在联调阶段表现良好是不够的，还需要为”人工介入时机”和”系统自身的异常恢复”设计同等重视的机制，这是本次实验相较前两次实验最重要的经验收获。

10 设计反思与改进展望

回顾整个设计过程，路线逻辑先行（第 3 节）、两层参数分离（第 4 节）、提前降速而非段边界切换（第 5 节）、责任范围内的避障特调（第 6 节）这四个决策对系统在联调阶段的性能贡献最大。仍存在的不足包括：参数调整目前主要依靠运动学估算与真实日志回溯（表 2），尚未做过系统化的多值对照实验；避障策略缺乏变道超车能力，且责任范围避障阈值本身也还没有真实多车数据验证、汇合区侧向特调尚未合并进主循环（第 6.2 节），这正是第 9 节问题一的根源；系统状态管理只考虑了”位置跳变”这一种异常情形，缺乏对”停止后恢复”与”冷启动”的区分，这是问题二的根源，根因尚待进一步排查，是报告完成后仍需继续跟进的开放问题。此外，Path 1 入弯的初始定位瞬态（第 3.1 节）提示起点定位与路段假设之间还存在需要对齐的细节。

11 实验总结

本次实验是三次实验中系统复杂度最高、也是设计思考最密集的一次。从单纯的算法实现，到涉及初始状态分析、路线策略、参数管理、弯道预处理、责任范围避障的完整系统，每一个组件都源于实测发现的真实问题。联调阶段的结果一度让笔者认为系统已经比较成熟，但对真实 `run_log.csv` 的深入分析、以及正式考试暴露出的两个问题——未及时手动超车导致被前车卡住、系统偶发停止后错误回退至路线 1 初始位置引发碰撞——说明”联调环境下表现良好”与”真实比赛环境下足够健壮”之间仍然存在差距：前者更多考验核心控制算法本身，后者还额外考验人机协作的介入时机设计、以及系统面对自身异常时的恢复能力。

落到最终成绩上，本次比赛笔者在 12 名参赛者中位列**第 5 名**（第 9.1 节表 4）：一方面，车辆里程跑满了封顶的 2000 m，说明路线规划、分段参数与弯道降速这套核心方案确实能让车”跟得住、跑得完”；另一方面，286.77 m 的扣分与 2 次违规也如实反映出系统在多车博弈、异常恢复上的短板。名次不是终点，而是把”哪里还不够稳”这件事量化了出来——如果能在下一版中补上主动超车与运行状态持久化这两块，笔者相信在里程同样跑满的前提下，扣分和违规都还有明显的下降空间。这也是这门课三次实验里，笔者收获最大的一次：真正的工程能力，不只在于让算法在顺境里跑得漂亮，更在于让系统在逆境里不至于崩盘。